

Anticipated Questions & Answers

Likely questions a supervisor or examiner may ask about this project, with grounded answers

This is preparation material, not a script to read aloud. Every answer below is grounded in the actual code and results in this repository (thesis_project/Step_01 through Step_10, and the backend/frontend source) so the numbers can be defended if questioned further. Read each answer, understand the reasoning behind it, and answer in your own words on the day.

A. Problem Statement & Dataset

1 Why predict hair-fall risk, and why frame it as a 3-class problem instead of regression?

The target (hair_fall) is a categorical severity/likelihood indicator, not a continuous measurement, so classification is the natural framing. Three classes — Low / Moderate / High — give a plain-language result a non-specialist can act on, and are the classes the dataset itself is labelled with.

2 What is the dataset and how large is it?

21,606 patient rows and 23 raw columns (Step 2), covering age, gender, seven blood-test measurements (total protein, calcium, iron, vitamin D, ALT liver enzyme, manganese, body water content), lifestyle/health flags (family history, chronic illness, sleep, water quality, chemical hair-product use, anaemia, stress) and the hair_fall label. There are zero missing values and zero duplicate rows out of the box.

3 Did you clean or filter the data, and how?

Step 3 checks every blood-test column against its clinically normal range and flags below-/above-range values (for example 39.8% of rows have vitamin D below the normal 25–80 range) — but these are kept, not dropped, because being outside a healthy range is exactly the kind of signal a hair-fall risk model should be able to use. A separate physiological-impossibility check (values outside what a living person could have) found 0 such rows, so no rows were removed. The identifying columns id and full_name are dropped before modelling; the final modelling table is 21,606 rows × 20 features.

4 Is the dataset balanced across classes?

No, and this is preserved intentionally: the class split is 45% Low, 35% Moderate, 20% High, and the train/test split (Step 4) is stratified so both sets keep exactly this ratio. Reporting macro-F1 alongside accuracy (see Section C) is how the evaluation accounts for the imbalance.

5 Is this real patient data? What about privacy?

The data has the shape of a de-identified/synthetic clinical-style dataset: each row has an id and a full_name field that is already stored in an obfuscated, non-plain-text form, not a directly readable name. Regardless of provenance, both id and full_name are dropped before any modelling (Step 3) and never sent to any model — only the 20 clinical/lifestyle features are used. The Risk-check tab's own patient entries stay entirely local (Postgres + MinIO on this machine, no cloud calls), which is covered further in Section H.

6 How was gender encoded, and does the small 'Other' category cause a problem?

Gender is label-encoded as Female=0, Male=1, Other=2 (Step 3) and treated by all three models as an ordinary numeric/categorical feature. The split is 12,109 Male, 9,485 Female and only 12 Other out of 21,606 rows — that category is too small (0.06%) for any model to learn a reliable pattern for it specifically, so predictions for those patients rely mostly on their other features. This is a real limitation for that small subgroup, discussed further in Section F.

7 Why include lifestyle and family-history flags alongside blood tests?

Hair fall is rarely explained by a single biomarker; family history, chronic illness, sleep quality, water quality, chemical product use and self-reported stress are all established contributing factors in the clinical literature, and including them lets the model (and the SHAP explanation) separate a biological signal (e.g. low iron) from a behavioural one (e.g. late-night sleep) for the same patient.

B. Experimental Design

1 Why train/test split 80/20, and why stratified?

17,284 training rows and 4,322 test rows (Step 4), an 80/20 split. Stratifying on the target keeps the 45/35/20 class ratio identical in both sets, so a model cannot look better or worse purely because the test set happened to get an easier class mix.

2 Why compare models at three different training-set sizes (500, 2000, Full)?

This is the central experimental variable of the thesis: it measures how each model's accuracy scales with the amount of data it sees, which matters because TabPFN and TabFM are in-context ("foundation") models that are not retrained per dataset — they condition on a context of example rows at inference time — so "training set size" for them literally means context size. CatBoost is retrained fully at each size for a fair comparison at the same data budget.

3 Why these three specific models — CatBoost, TabPFN, TabFM?

CatBoost is a strong, fast, well-understood gradient-boosted tree baseline that is normally trained per dataset. TabPFN and TabFM are both pretrained tabular foundation models that use in-context learning instead of dataset-specific training — but from two different model families/vendors — so comparing all three tests whether a foundation-model approach can match a traditional trained model on this specific clinical-style tabular task, and whether the two foundation models behave similarly to each other.

4 What does “context size” mean for TabPFN/TabFM, concretely?

Instead of updating internal weights, these models are given up to a few thousand labelled example rows at prediction time as context, and predict the new row's class from patterns in that context (similar in spirit to a very large k-nearest-neighbours lookup executed by a neural network). Step 4 draws the same seeded, stratified 500- and 2000-row samples for both models, and “Full” is the entire 17,284-row training set.

5 How were CatBoost's hyperparameters chosen — was it tuned fairly?

Yes: for every training size, Step 5 runs a full grid search over tree depth {4, 6, 8} × learning rate {0.01, 0.03, 0.1} (9 combinations) using 5-fold stratified cross-validation on the training rows only, selecting the combination with the highest mean CV macro-F1, with early stopping (50 rounds) inside each fold to pick the iteration count. Only after that selection is the final model refit on all training rows and evaluated once on the untouched test set — the test set is never used for model or hyperparameter selection at any point.

6 Why cross-validate CatBoost's hyperparameters but use a single train/test split for the final comparison?

These answer different questions. The 5-fold CV inside Step 5 is only for choosing CatBoost's own depth/learning-rate — it never touches the test set. The final cross-model comparison (Step 10) needs every model scored on the exact same held-out rows so that McNemar's paired test (Section C) is valid; that requires one fixed test set, not a rotating CV fold, since McNemar's test compares two models' correct/incorrect calls on the identical set of examples.

7 Why CatBoost specifically, rather than XGBoost, LightGBM or Random Forest?

CatBoost was chosen as the “conventional, trained-per-dataset” baseline representative of gradient-boosted trees generally; it handles the categorical/boolean features in this dataset natively without manual one-hot encoding, trains quickly enough to grid-search at every size, and is a fair, strong, widely-used comparison point for what a foundation model would need to beat. The thesis's contribution is the comparison methodology (accuracy scaling with data size, paired significance testing, explainability cost) rather than an exhaustive search over every possible tree-based baseline.

8 Why not train a deep learning model (e.g. a plain neural network) from scratch as another baseline?

Deep learning models trained from scratch typically need far more labelled rows than are available at the 500–2000-row sizes this thesis specifically studies to outperform tree-based methods on tabular data — this is precisely the data-scarcity gap that pretrained tabular foundation models (TabPFN, TabFM) are designed to close. Including one would answer a different question (“how much data does a from-scratch network need”) than the one this thesis asks (“does in-context learning beat a trained model at a given data budget”).

C. Results & Statistical Testing

1 What were the headline results?

From Step 10's final comparison table (test set of 4,322 rows, held constant across all nine model×size combinations):

- At size 500: TabPFN 76.89% accuracy / 0.9159 ROC-AUC, TabFM 76.84% / 0.9151, CatBoost only 69.37% / 0.8611.
- At size 2000: TabFM 78.44% / 0.9209, TabPFN 78.27% / 0.9220, CatBoost 74.97% / 0.9029.
- At Full (17,284 rows): TabFM 78.67% / 0.9233, TabPFN 78.62% / 0.9235, CatBoost 78.20% / 0.9197.

The pattern is consistent: the foundation models have a clear edge with little data, and CatBoost closes most of that gap once it sees the full training set.

2 Is the difference between models actually statistically significant, or could it be noise?

This is tested directly with McNemar's test on paired predictions (Step 9), which is the appropriate test here because all three models are scored on the exact same 4,322 test rows. Results: CatBoost vs TabPFN and CatBoost vs TabFM are both highly significant at size 500 and 2000 ($p < 0.001$ in all four comparisons), but NOT significant at Full training size ($p = 0.27$ and $p = 0.23$) — meaning with enough data, CatBoost's remaining gap to the foundation models is not statistically distinguishable from chance. TabPFN vs TabFM is never significant at any size ($p > 0.59$ throughout) — the two foundation models perform statistically equivalently to each other at every data size tested.

3 What is the practical implication of the McNemar's result?

If a lot of labelled data is available, a conventional trained model (CatBoost) is a perfectly competitive choice and is far cheaper to run (fits in ~3.6s vs 15s, and predicts a full test set instantly vs. TabFM's 3,082 seconds at Full — see the next section). The foundation models' real advantage in this thesis is in the low-data regime (500–2000 rows), where they significantly outperform a freshly trained tree model.

4 Which class is hardest to predict, and what do the errors look like?

Moderate risk is consistently the weakest class across every model and size — for example CatBoost at Full scores F1 0.854 on Low and 0.769 on High, but only 0.701 on Moderate (Step 5c's classification report; the same pattern holds for TabPFN and TabFM). This is expected: Moderate sits between the other two classes with no hard boundary in the underlying biology, so it is the class most often confused with its neighbours, while Low and High — the more clinically distinct extremes — are easier to separate.

5 Are the models overfitting? How do you know?

Three checks make overfitting unlikely to explain the results. First, CatBoost's hyperparameters are chosen by cross-validation macro-F1 (e.g. 0.7669 at Full) and then evaluated once on a completely untouched test set, where it scores a comparable 0.7744 macro-F1 — a held-out score this close to the CV score is a sign of good generalisation, not overfitting (a large drop would be the red flag). Second, TabPFN/TabFM never update weights on this dataset at all — they condition on context at inference time, so classical overfitting-to-training-data in the usual sense does not apply to them the same way. Third, early stopping (50 rounds) is used inside every CatBoost CV fold specifically to stop training before it starts memorising the fold.

D. Explainability (SHAP)

1 How are the models explained, and why does it differ by model?

CatBoost uses exact TreeSHAP, which is fast because the tree structure gives closed-form Shapley values: explaining 1,000 rows takes about 0.3 seconds regardless of training size. TabPFN and TabFM are not tree models, so they need model-agnostic KernelSHAP, which is dramatically slower — Step 10 recorded 65.8s for just 30 TabPFN rows at size 500, rising to 3,251s at Full; for TabFM, 158s for 20 rows at 500 rising to 8,491s (about 2.4 hours) at Full.

2 Given that cost, how does the live Risk-check tab show “why” for a single patient?

It uses CatBoost's exact TreeSHAP only, computed instantly for that one patient row, and reports the difference between the SHAP value for the High-risk class and the Low-risk class per feature as that feature's risk direction. TabPFN/TabFM's KernelSHAP is only ever computed offline, in bulk, for the Results tab — never per-patient — because it would make a live check take minutes to hours.

3 What is the actual difference between TreeSHAP and KernelSHAP?

TreeSHAP computes exact Shapley values by walking the tree structure directly — it is a closed-form algorithm specific to tree ensembles, which is why it is both exact and fast for CatBoost. KernelSHAP is model-agnostic: it treats the model as a black box and estimates Shapley values by repeatedly querying it with perturbed inputs and fitting a weighted linear model to the results, which is necessary for TabPFN/TabFM (not tree-based) but is only an approximation and needs many model calls per row — the source of its far higher cost.

4 Which features matter most, and does that make clinical sense?

Iron and stress_level are the top two SHAP-ranked features across almost every model and size (Step 10's top_5 column), followed by total_protein, calcium and alt_liver. This lines up with the clinical literature: iron-deficiency and chronic stress are well-documented contributors to hair shedding, which is a useful sanity check that the models are picking up a real biological signal rather than an artefact of the data.

E. System Design & Engineering

1 Why train on Kaggle instead of the laptop that runs the web app?

The laptop is CPU-only; TabPFN/TabFM fitting, and especially TabFM's SHAP computation (up to 2.4 hours per size), need a GPU to be practical. Kaggle offers a free GPU notebook environment, so the heavy, one-off training/explaining work is done there once, and only its outputs (metrics, figures, the trained CatBoost model file, logs) are brought back to the machine that runs the app.

2 How do Kaggle's results get into your local app? Walk me through it.

Each Kaggle step script writes a result folder (output.txt log plus metrics/model/figure files), which is downloaded as a zip to ~/Documents. A small importer (backend/import_kaggle_run.py) scans that folder on every app start, matches each zip to a step by id, and pairs the Kaggle output with the LOCAL copy of that step's code (so it stays re-runnable). It inserts a row into Postgres, uploads files to MinIO, and mirrors everything to a plain thesis_project/<Step>/ folder on disk. The web app's Notebook tab then just displays whichever run — Kaggle or local — is newest for each step. (See the companion “System Architecture” document for the full diagram.)

3 Why Postgres and MinIO instead of just files, or a simpler database?

Postgres gives structured, queryable history for every run and every prediction (status, timing, which code produced which output), which the UI needs to list, filter and reopen past results quickly. MinIO is an S3-compatible object store used for the actual files (images, model binaries, CSVs) that don't belong in relational rows; both run locally via Docker, so the whole system is self-contained and needs no cloud account.

4 Why FastAPI and a plain HTML/JS frontend rather than a framework like React?

The app is a personal thesis tool with four fairly simple views (Notebook, Risk check, Results, Dataset) talking to a REST backend; FastAPI gives fast, typed endpoints with minimal boilerplate, and plain HTML/CSS/JS avoids a build step, keeping the whole thing runnable with one script and easy to read for anyone reviewing the code.

5 The Risk-check tab predicts a live patient using models trained for the thesis — how does that work?

It reuses the exact artifacts Steps 4–7 produced: the Step 4 training rows as TabPFN/TabFM's in-context examples, and the Step 5c CatBoost model file. A submitted patient row is run through all three models in a background thread, their class probabilities are soft-voting averaged into a single Low/Moderate/High result and confidence score, and the result plus a full audit trail is saved to Postgres and mirrored to MinIO as JSON.

6 A single live prediction was reportedly slow (minutes) at first — why, and how was it fixed?

TabFM has no attention/context cache: every single-patient prediction re-runs a full forward pass over its whole context, and that cost scales with context size, not with how many rows are being predicted (this is visible in Step 10 too — TabFM's `predict_seconds` for a 4,322-row batch is barely different from the cost of one row, because the context, not the query, dominates). At the original default context size (2000 rows) this took roughly 145 seconds per patient on a CPU-only machine, pushing a full three-model check to 3–4 minutes. The default was changed to a 500-row context (about 4× less compute) for roughly 1.6 points less ROC-AUC than 2000 (0.9151 vs 0.9209, per Step 7), cutting a live check to about 35 seconds end-to-end. TabPFN, by contrast, does support an attention cache (`fit_with_cache`), so its single-patient cost is under a second once warmed — no accuracy trade-off was needed there. A live estimated-time-remaining indicator was also added, computed from each model's own most recent run time, so the person checking sees real progress instead of an unexplained wait.

7 Why average all three models' predictions (soft voting) instead of just using the single best one?

No single model wins outright at every size (TabPFN is best at 500, TabFM at 2000 and Full, by very small margins — and Section C's McNemar's test shows TabPFN and TabFM are never significantly different from each other). Averaging the three models' probabilities is a simple ensembling approach that reduces variance from any one model's occasional mistake, and the UI's agreement flag (whether all three models picked the same class) doubles as a built-in, visible confidence signal for the person reading the result.

8 What happens when the three models disagree with each other?

The consensus is still the class with the highest averaged probability, but the report explicitly says “the models did not fully agree” instead of claiming unanimous agreement, and the patient-facing view adds a note when the SHAP-attributed risk factors point in genuinely mixed directions (some raising, some lowering risk) so the person is not given a false sense of certainty.

9 Why save every Risk-check permanently instead of just showing the result and discarding it?

Persisting every check (to Postgres, mirrored to MinIO as JSON) makes results reproducible and auditable — a check can be reopened later with its exact inputs, per-model probabilities and SHAP breakdown intact, which matters both for demonstrating the system in a defence and, in a real deployment, for a clinician or the patient to revisit a past result rather than trusting memory of it.

F. Limitations & Validity

1 What are the main limitations of this work?

(1) A single dataset with no external validation cohort, so generalisation beyond this data's population and measurement conventions is untested. (2) The three training sizes (500/2000/Full) all come from the same 17,284-row pool via stratified sampling, not independent samples, so the scaling curve reflects one draw at each size. (3) TabFM's live single-patient cost still scales with context size with no caching available in the current library version, which is why the deployed default uses a smaller context than the best-scoring configuration. (4) The tool is explicitly a research estimate, not a diagnostic device — this is stated in the UI itself.

2 Why is this not a medical diagnosis tool?

The models are trained on one dataset's statistical associations between lab/lifestyle values and a labelled hair-fall outcome; they cannot account for a clinician's full context, comorbidities, or causes outside the recorded features. The Risk-check report says this explicitly and recommends discussing any concerning or out-of-range result with a doctor.

3 Could the model be biased against any group, given the gender imbalance in the data?

This is a genuine, disclosed limitation. With only 12 ‘Other’-gender rows out of 21,606, no per-group accuracy breakdown is statistically meaningful for that subgroup, and the model has essentially learned its behaviour for Male/Female patients only. A fairness audit — accuracy and calibration broken down by gender — is listed as follow-up work (Section G) rather than something already verified.

G. Data Ethics, Privacy & Reproducibility

1 Is patient data from the Risk-check tab kept secure?

Yes by construction, not just policy: the whole stack (FastAPI, Postgres, MinIO) runs on 127.0.0.1 on the user's own machine, started by a local script, with no external API calls made with patient data at any point — predictions run entirely on locally-loaded model files. There is no cloud upload of any check someone enters.

2 Are there licensing considerations for using TabPFN or TabFM?

Yes — TabPFN requires accepting its license (TabPFN-3.5) once via the Prior Labs account before it can be used, which is a one-time setup step documented in the project README; this is a usage-license acceptance rather than a cost, but it is worth being able to name if asked why an API key/login step exists in the setup at all.

3 Are your results reproducible? What would someone need to redo this?

Every stochastic step is seeded (SEED = 42 throughout: the train/test split, the stratified context sampling, CatBoost's CV folds and final fit, TabPFN/TabFM's random state), so re-running a step script against the same `thesis_project/data/data.csv` reproduces the same rows and, for CatBoost, numerically the same model. TabPFN/TabFM's exact floating-point outputs can vary slightly by hardware/driver version since they run on GPU, but the same context rows and same code are used regardless of where it runs.

H. Possible Follow-up Questions

1 What would you do differently, or next, given more time?

Validate on an independent external dataset; test calibration of the predicted probabilities, not just accuracy; run a fairness audit by gender given the 'Other'-category imbalance noted in Section F; investigate whether TabFM's library exposes any caching mechanism for repeated single-row queries against a fixed context (analogous to TabPFN's `fit_with_cache`) to remove the accuracy/speed trade-off currently made for live use; and extend McNemar's pairwise testing with a multi-model omnibus test (e.g. Cochran's Q) as a robustness check.

2 Why report both accuracy and macro-F1?

Accuracy alone can look inflated on an imbalanced label set (45/35/20 split here) by favouring the majority class; macro-F1 averages the F1 score across all three classes unweighted, which is a fairer read on how well the Moderate and High classes — the ones a risk tool most needs to get right — are actually being predicted.

3 How would you take this from a thesis prototype to something used in a real clinical setting?

At minimum: external validation on a second, independently collected cohort; a calibration study (do predicted probabilities match observed outcome rates); a fairness audit across gender and age groups; clinical sign-off on the feature list and thresholds; and almost certainly a hosted, access-controlled deployment in place of the current local-machine setup, with proper patient-consent and data-retention policies rather than the indefinite local storage used for this thesis prototype.

Source of every figure above: `thesis_project/Step_02_LoadData`, `Step_03_CleanEncode`, `Step_04_TrainTestSplit`, `Step_05a_CatBoost_500` (`grid_search.csv`), `Step_07a/b/c_TabFM_*`, `Step_09_McNemarsTests` and `Step_10_FinalResults` (`output.txt` / `metrics.json` in each folder), plus `backend/app/predict.py` and `backend/import_kaggle_run.py`.